In [ ]:
```
#%pip install tensorflow==2.15.0
#%pip install scikit-learn
#%pip install numpy
#%pip install matplotlib
#%pip install pandas
#%pip install seaborn
```

In [ ]:
```
#%pip install scikeras
#%pip install --upgrade pip
#%pip install tensorflow scikit-learn scikeras numpy matplotlib pandas seaborn
```

In [ ]:
```
import sys
print(sys.version)
print(sys.executable)
```

In [1]:
```
# Evaluation library
from sklearn.metrics import confusion_matrix, accuracy_score
from sklearn.model_selection import GridSearchCV

# Deep Learning libraries
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Activation, Dropout
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical

# Scikit-learn wrapper for Keras
from scikeras.wrappers import KerasClassifier
```

In [2]:
```
# Load MNIST handwritten digit dataset
(X_train_digit, y_train_digit), (X_test_digit, y_test_digit) = mnist.load_data()

print("Training images:", X_train_digit.shape)
print("Training labels:", y_train_digit.shape)
print("Testing images:", X_test_digit.shape)
print("Testing labels:", y_test_digit.shape)
```

```
Training images: (60000, 28, 28)
Training labels: (60000,)
Testing images: (10000, 28, 28)
Testing labels: (10000,)
```

In [3]:
```
X_train_digit
```

Out[3]:
```
array([[[0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0],
        ...,
        [0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0]],

       [[0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0],
        ...,
        [0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0]],

       [[0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0],
        ...,
        [0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0]],

       ...,

       [[0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0],
        ...,
        [0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0]],

       [[0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0],
        ...,
        [0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0]],

       [[0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0],
        ...,
        [0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0]]], dtype=uint8)
```

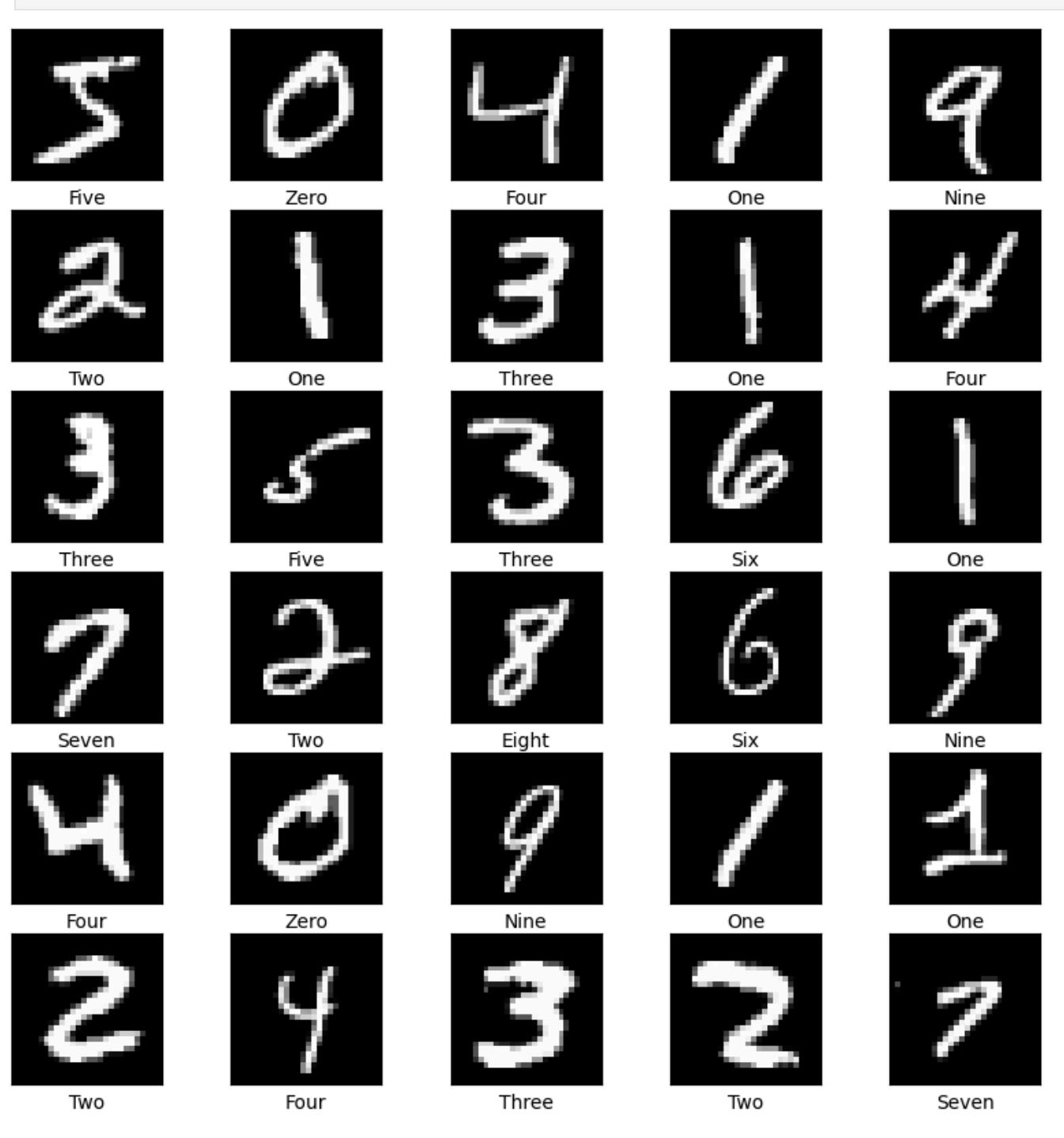
In [33]:
```
y_train_digit
```

Out[33]:
```
array([[0., 0., 0., ..., 1., 0., 0.],
       [0., 0., 1., ..., 0., 0., 0.],
       [0., 1., 0., ..., 0., 0., 0.],
       ...,
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.]])
```

In [5]:
```
 X_train_digit[1]
```

Out[5]:
```
array([[  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0,  51, 159, 253, 159,  50,   0,   0,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,  48, 238, 252, 252, 252, 237,   0,   0,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
         54, 227, 253, 252, 239, 233, 252,  57,   6,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,  10,  60,
        224, 252, 253, 252, 202,  84, 252, 253, 122,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0, 163, 252,
        252, 252, 253, 252, 252,  96, 189, 253, 167,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,  51, 238, 253,
        253, 190, 114, 253, 228,  47,  79, 255, 168,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0,  48, 238, 252, 252,
        179,  12,  75, 121,  21,   0,   0, 253, 243,  50,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,  38, 165, 253, 233, 208,
         84,   0,   0,   0,   0,   0,   0, 253, 252, 165,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   7, 178, 252, 240,  71,  19,
         28,   0,   0,   0,   0,   0,   0, 253, 252, 195,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,  57, 252, 252,  63,   0,   0,
          0,   0,   0,   0,   0,   0,   0, 253, 252, 195,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0, 198, 253, 190,   0,   0,   0,
          0,   0,   0,   0,   0,   0,   0, 255, 253, 196,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,  76, 246, 252, 112,   0,   0,   0,
          0,   0,   0,   0,   0,   0,   0, 253, 252, 148,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,  85, 252, 230,  25,   0,   0,   0,
          0,   0,   0,   0,   0,   7, 135, 253, 186,  12,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,  85, 252, 223,   0,   0,   0,   0,
          0,   0,   0,   0,   7, 131, 252, 225,  71,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,  85, 252, 145,   0,   0,   0,   0,
          0,   0,   0,  48, 165, 252, 173,   0,   0,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,  86, 253, 225,   0,   0,   0,   0,
          0,   0, 114, 238, 253, 162,   0,   0,   0,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,  85, 252, 249, 146,  48,  29,  85,
        178, 225, 253, 223, 167,  56,   0,   0,   0,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,  85, 252, 252, 252, 229, 215, 252,
        252, 252, 196, 130,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,  28, 199, 252, 252, 253, 252, 252,
        233, 145,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,  25, 128, 252, 253, 252, 141,
         37,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0]], dtype=uint8)
```

In [9]:
```
import matplotlib.pyplot as plt

# Names of numbers in the dataset in order
col_names = [
    'Zero', 'One', 'Two', 'Three', 'Four',
    'Five', 'Six', 'Seven', 'Eight', 'Nine'
]

# Visualizing the digits
plt.figure(figsize=(10, 10))

for i in range(30):
    plt.subplot(6, 5, i + 1)
    plt.xticks([])
    plt.yticks([])
    plt.imshow(X_train_digit[i], cmap='gray')
    plt.xlabel(col_names[y_train_digit[i]])

plt.show()
```



In [10]:
```
X_train_digit.shape
```

Out[10]:
```
(60000, 28, 28)
```

In [11]:
```
X_test_digit.shape
```

Out[11]:
```
(10000, 28, 28)
```

In [13]:
```
X_train_digit = X_train_digit.reshape(60000, 784)
X_test_digit = X_test_digit.reshape(10000, 784)
```

In [14]:
```
X_train_digit.shape
```

Out[14]:
```
(60000, 784)
```

In [15]:
```
X_train_digit
```

Out[15]:
```
array([[0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       ...,
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0]], dtype=uint8)
```

In [16]:
```
y_train_digit.shape
```

Out[16]:
```
(60000,)
```

In [17]:
```
# Encoding Digit MNIST Labels
y_train_digit = to_categorical(y_train_digit, num_classes=10)

y_test_digit = to_categorical(y_test_digit, num_classes=10)
```

In [18]:
```
y_train_digit.shape
```

Out[18]:
```
(60000, 10)
```

In [19]:
```
y_train_digit
```

Out[19]:
```
array([[0., 0., 0., ..., 0., 0., 0.],
       [1., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       ...,
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 1., 0.]])
```

In [20]:
```
y_test_digit
```

Out[20]:
```
array([[0., 0., 0., ..., 1., 0., 0.],
       [0., 0., 1., ..., 0., 0., 0.],
       [0., 1., 0., ..., 0., 0., 0.],
       ...,
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.]])
```

In [22]:
```
# Creating base neural network
model = keras.Sequential([
    layers.Dense(256, activation='relu', input_shape=(784,)),
    # layers.Dropout(0.3),
    # layers.BatchNormalization(),

    layers.Dense(64, activation='relu'),
    # layers.Dropout(0.3),
    # layers.BatchNormalization(),

    layers.Dense(64, activation='relu'),
    # layers.Dropout(0.3),
    # layers.BatchNormalization(),

    layers.Dense(10, activation='sigmoid'),
])
```

```
C:\Users\fazil\anaconda3\Lib\site-packages\keras\src\layers\core\dense.py:87: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

In [23]:
```
# Compiling the model
model.compile(
    loss="categorical_crossentropy",
    optimizer="adam",
    metrics=['accuracy']
)
```

In [24]:
```
# Training the model
history = model.fit(
    X_train_digit,
    y_train_digit,
    batch_size=100,
    epochs=11,
    validation_data=(X_test_digit, y_test_digit)
)
```

```
Epoch 1/10
600/600 ━━━━━━━━━━━━━━━━━━━━ 9s 8ms/step - accuracy: 0.7771 - loss: 3.9859 - val_accuracy: 0.9192 - val_loss: 0.3426
Epoch 2/10
600/600 ━━━━━━━━━━━━━━━━━━━━ 6s 9ms/step - accuracy: 0.9340 - loss: 0.2630 - val_accuracy: 0.9357 - val_loss: 0.2431
Epoch 3/10
600/600 ━━━━━━━━━━━━━━━━━━━━ 7s 11ms/step - accuracy: 0.9551 - loss: 0.1572 - val_accuracy: 0.9536 - val_loss: 0.1870
Epoch 4/10
600/600 ━━━━━━━━━━━━━━━━━━━━ 6s 11ms/step - accuracy: 0.9663 - loss: 0.1168 - val_accuracy: 0.9455 - val_loss: 0.2178
Epoch 5/10
600/600 ━━━━━━━━━━━━━━━━━━━━ 6s 9ms/step - accuracy: 0.9662 - loss: 0.1145 - val_accuracy: 0.9457 - val_loss: 0.2134
Epoch 6/10
600/600 ━━━━━━━━━━━━━━━━━━━━ 6s 10ms/step - accuracy: 0.9704 - loss: 0.1004 - val_accuracy: 0.9649 - val_loss: 0.1490
Epoch 7/10
600/600 ━━━━━━━━━━━━━━━━━━━━ 6s 9ms/step - accuracy: 0.9758 - loss: 0.0822 - val_accuracy: 0.9625 - val_loss: 0.1601
Epoch 8/10
600/600 ━━━━━━━━━━━━━━━━━━━━ 6s 10ms/step - accuracy: 0.9773 - loss: 0.0774 - val_accuracy: 0.9583 - val_loss: 0.1896
Epoch 9/10
600/600 ━━━━━━━━━━━━━━━━━━━━ 9s 8ms/step - accuracy: 0.9807 - loss: 0.0655 - val_accuracy: 0.9618 - val_loss: 0.1595
Epoch 10/10
600/600 ━━━━━━━━━━━━━━━━━━━━ 5s 8ms/step - accuracy: 0.9802 - loss: 0.0684 - val_accuracy: 0.9636 - val_loss: 0.1556
```

In [25]:
```
# Predicting the labels - DIGIT
y_predict = model.predict(X_test_digit)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
```

In [28]:
```
y_predict
```

Out[28]:
```
array([[3.01682979e-01, 1.69056933e-02, 4.80215341e-01, ...,
        1.00000000e+00, 1.66296129e-04, 4.72181380e-01],
       [2.90187558e-07, 7.91750073e-01, 1.00000000e+00, ...,
        9.98089672e-06, 7.34739304e-01, 2.02559522e-06],
       [1.19542725e-01, 9.99999046e-01, 9.15602028e-01, ...,
        9.98814940e-01, 9.87784088e-01, 9.50060308e-01],
       ...,
       [1.06860854e-04, 9.99999702e-01, 9.99967933e-01, ...,
        9.99998987e-01, 9.99998987e-01, 9.99999702e-01],
       [9.46959198e-01, 1.72956176e-02, 5.90939373e-02, ...,
        8.39560986e-01, 9.98076677e-01, 3.16473931e-01],
       [9.44306791e-01, 1.26496516e-07, 1.07007496e-01, ...,
        9.99997556e-01, 5.00437498e-01, 9.19313967e-01]], dtype=float32)
```

In [29]:
```
y_predict[0]
```

Out[29]:
```
array([3.0168298e-01, 1.6905693e-02, 4.8021534e-01, 9.9999982e-01,
       8.1018937e-01, 3.8342972e-03, 6.2733261e-06, 1.0000000e+00,
       1.6629613e-04, 4.7218138e-01], dtype=float32)
```

In [30]:
```
import numpy as np
# Get the index of the maximum value in the predicted vector
y_predicts = np.argmax(y_predict, axis=1)
```

In [32]:
```
y_predicts[0]
```

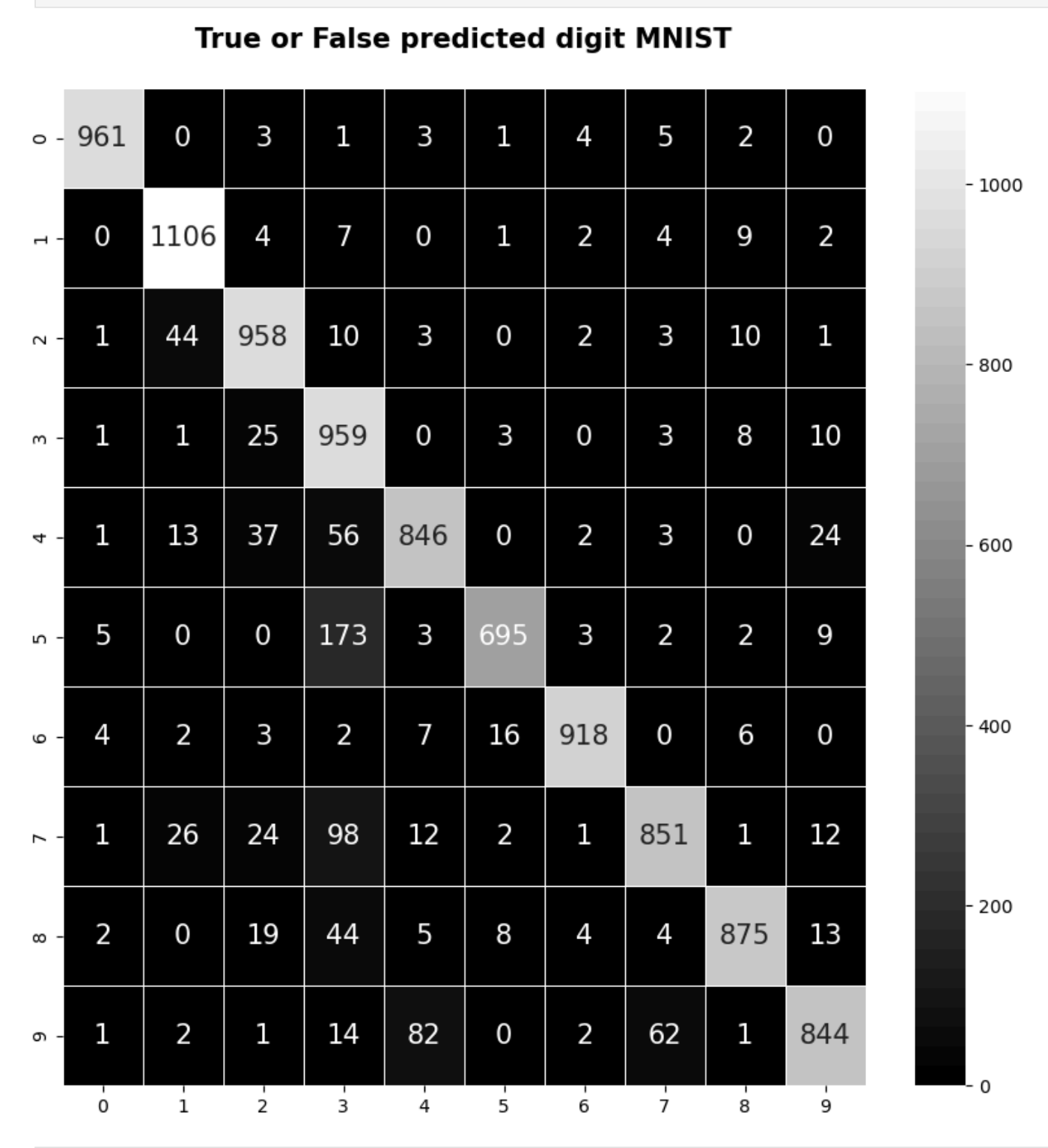
Out[32]:
```
7
```

In [34]:
```
# Get the actual labels from the one-hot encoded vectors
y_test_digit_eval = np.argmax(y_test_digit, axis=1)
```

In [35]:
```
y_test_digit_eval
```

Out[35]:
```
array([7, 2, 1, ..., 4, 5, 6], dtype=int64)
```

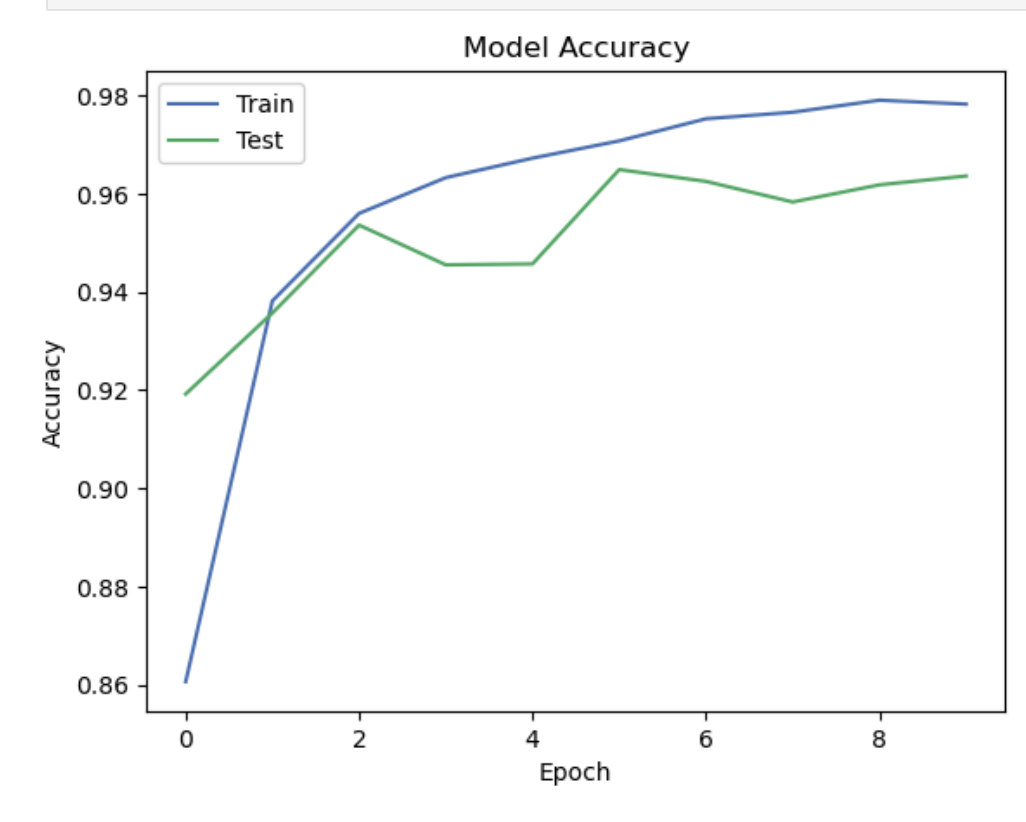
In [36]:
```
import seaborn as sns
# Confusion matrix for Digit MNIST
con_mat = confusion_matrix(y_test_digit_eval, y_predicts)

plt.style.use('seaborn-v0_8-deep')
plt.figure(figsize=(10, 10))

sns.heatmap(
    con_mat,
    annot=True,
    annot_kws={'size': 15},
    linewidths=0.5,
    fmt='d',
    cmap='gray'
)

plt.title(
    'True or False predicted digit MNIST\n',
    fontweight='bold',
    fontsize=15
)

plt.show()
```



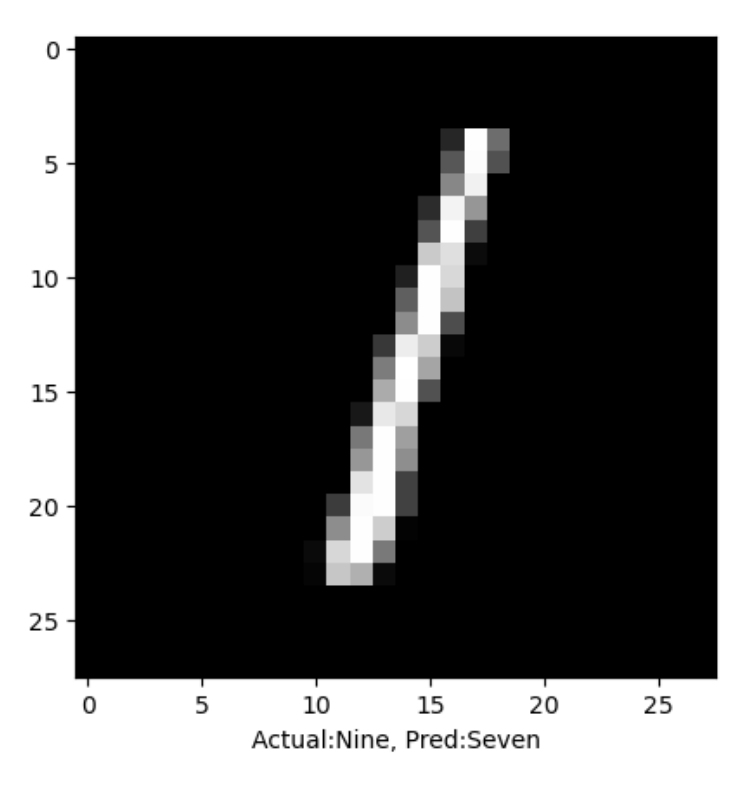
In [38]:
```
from sklearn.metrics import classification_report

print(classification_report(y_test_digit_eval, y_predicts))
```

```
              precision    recall  f1-score   support

           0       0.98      0.98      0.98       980
           1       0.93      0.97      0.95      1135
           2       0.89      0.93      0.91      1032
           3       0.70      0.95      0.81      1010
           4       0.88      0.86      0.87       982
           5       0.96      0.78      0.86       892
           6       0.98      0.96      0.97       958
           7       0.91      0.83      0.87      1028
           8       0.96      0.90      0.93       974
           9       0.92      0.84      0.88      1009

    accuracy                           0.90     10000
   macro avg       0.91      0.90      0.90     10000
weighted avg       0.91      0.90      0.90     10000
```

In [37]:
```
# Summarize history for accuracy
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])

plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')

plt.legend(['Train', 'Test'], loc='best')
plt.show()
```



In [42]:
```
# Names of numbers in the dataset in order
col_names = [
    'Zero', 'One', 'Two', 'Three', 'Four',
    'Five', 'Six', 'Seven', 'Eight', 'Nine'
]

# Visualizing the digit
plt.imshow(X_test_digit[2].reshape(28, 28), cmap='gray')

plt.xlabel(
    "Actual:{}, Pred:{}".format(
        col_names[y_test_digit_eval[7]],
        col_names[y_predicts_single[0]]
    )
)

plt.show()
```



```
In [ ]:
```